

1. Scenario

An e-commerce application allows customers to open a category page such as Electronics, Cameras, or Home & Kitchen. A category may contain several levels of subcategories, so a customer browsing a parent category should see products assigned to that category and to every category below it.

The browse engine must also respect the customer's region. A US request must not expose listings intended for India, and an India request must not include US-only listings. After filtering, the products are ranked by popularity so that the most relevant items appear first.

2. The Problem

The current implementation is incorrect. Depending on the request, it may omit descendants, include products from unrelated categories, leak products from another region, parse product fields incorrectly, or return items in the wrong order.

Your task

Inspect the existing project and repair the implementation so that every browse request returns exactly the eligible products, in the required order and output format. Do not redesign the public interfaces unless necessary.

3. Expected Processing Flow

1. Collect the requested category and every descendant beneath it.
2. Keep only products whose category belongs to that collected subtree.
3. Keep only products whose region matches the request exactly.
4. Record the total number of eligible products before limiting the output.
5. Sort eligible products by popularity score, using product ID as the tie-breaker.
6. Return at most the requested number of products.

4. Input Files

4.1 Category hierarchy - data/categories.json

This file contains a JSON object with a flat array named `categories`. Each category record contains:

- `category_id` - unique category identifier
- `name` - display name
- `parent_id` - ID of the parent category; null means the category is top-level

```
{
  "categories": [
    {"category_id": "ELEC", "name": "Electronics", "parent_id": null},
    {"category_id": "ELEC-MOB", "name": "Mobile Devices", "parent_id": "ELEC"},
    {"category_id": "ELEC-MOB-UNL", "name": "Unlocked Phones", "parent_id": "ELEC-MOB"},
    {"category_id": "HOME", "name": "Home & Kitchen", "parent_id": null}
  ]
}
```

Top-level categories form separate trees. Browsing `ELEC` must never include products from `HOME` or any category below it.

4.2 Product listings - data/products.jsonl

This is a JSON Lines file: each line is one complete JSON object. Every product contains:

- `product_id` - unique product identifier
- `name` - product display name
- `category_id` - category to which the listing is assigned
- `region` - region in which the listing is visible
- `popularity_score` - integer used for ranking; larger values rank higher

```
{"product_id": "P1001", "name": "Galaxy Nova 12", "category_id": "ELEC-MOB-
```

```
UNL", "region": "US", "popularity_score": 92}
{"product_id": "P1002", "name": "Pixel Horizon", "category_id": "ELEC-MOB-
UNL", "region": "IN", "popularity_score": 95}
{"product_id": "P1003", "name": "DoorGuard Camera", "category_id": "ELEC-SEC-
CAM", "region": "US", "popularity_score": 88}
```

4.3 Browse requests - data/requests.jsonl

Each line represents one request and contains:

- query_id - unique request identifier
- category_id - category being browsed
- region - exact region filter
- max_results - optional maximum number of products to return; defaults to 5

```
{"query_id": "q1", "category_id": "ELEC", "region": "US", "max_results": 3}
{"query_id": "q2", "category_id": "ELEC-MOB", "region": "IN"}
```

5. Output File

Write the results to `data/results.json` as a JSON array. Results must appear in the same order as the input requests.

Each result object contains:

- query_id - copied from the request
- matched_count - number of eligible products after category and region filtering, before max_results is applied
- products - up to max_results ranked products; omit the region field from each returned product

```
[
  {
    "query_id": "q1",
    "matched_count": 2,
    "products": [
      {"product_id": "P1001", "name": "Galaxy Nova 12", "category_id": "ELEC-MOB-
UNL", "popularity_score": 92},
      {"product_id": "P1003", "name": "DoorGuard Camera", "category_id": "ELEC-SEC-
CAM", "popularity_score": 88}
    ]
  }
]
```

6. Decision Rules

Step 1 - Build the included category set

Starting from the request's `category_id`, include:

- the requested category itself
- all children, grandchildren, and deeper descendants

Unknown category

If the requested category ID does not exist in the hierarchy, return `matched_count = 0` and an empty products array.

Step 2 - Filter eligible products

A product is eligible only when both conditions are true:

- its `category_id` is in the included category set
- its `region` is exactly equal to the request region; matching is case-sensitive

After this filtering step, set `matched_count` to the number of remaining products.

Step 3 - Rank and limit

Sort eligible products using the following keys, in order:

1. popularity_score descending - highest score first
2. product_id ascending - used only when popularity scores are equal

Return the first `max_results` products. When the field is absent or null, use a default value of 5.

7. Worked Examples

The following examples use a small illustrative catalog. They demonstrate the required behavior and are independent of any larger dataset supplied with the project.

Example 1 - A top-level category includes every descendant

```
Request
{"query_id":"q001","category_id":"ELEC","region":"US","max_results":3}

Eligible US products under ELEC
P1001  score 97  category ELEC-MOB-UNL
P1010  score 94  category ELEC-CAM-MIR
P1008  score 92  category ELEC-CAM-DSLR
P1023  score 99  category HOME-KIT    <-- excluded: different tree

Result
{
  "query_id":"q001",
  "matched_count":3,
  "products":["P1001","P1010","P1008"]
}
```

Example 2 - Region filtering removes cross-region listings

```
Request
{"query_id":"q005","category_id":"ELEC-MOB-UNL","region":"IN","max_results":5}

Listings in this category
P1001  region US  score 97  <-- excluded
P1002  region US  score 90  <-- excluded
P1003  region IN  score 96
P1004  region IN  score 99

Result order: P1004, P1003
matched_count: 2
```

Example 3 - A middle category includes only its own subtree

```
Request
{"query_id":"q008","category_id":"ELEC-CAM","region":"US","max_results":3}

Included categories
ELEC-CAM
ELEC-CAM-DIG
ELEC-CAM-DSLR
ELEC-CAM-MIR
ELEC-CAM-VID
ELEC-CAM-ACT

Products from ELEC-MOB or ELEC-SEC are siblings and must not be included.
```

Example 4 - max_results is optional

```
Request
{"query_id":"q028","category_id":"ELEC-MOB","region":"IN"}

Because max_results is missing, the engine uses 5.
```

If only 3 products are eligible, `matched_count` is 3 and all 3 are returned.

8. Project Layout

```
include/  
  browse/  
    models.hpp      Data models  
    parser.hpp      JSON parsing declarations  
    engine.hpp      Browse engine interface  
    file_io.hpp     File input/output declarations  
src/  
  browse/  
    parser.cpp      JSON parsing implementation  
    engine.cpp      Category traversal, filtering, ranking  
    file_io.cpp     File reading and result writing  
    main.cpp        Command-line entry point  
tests/  
  engine_test.cpp  
data/  
  categories.json  
  products.jsonl  
  requests.jsonl  
run.sh  
test.sh  
CMakeLists.txt  
README.md
```

9. What Your Fix Must Satisfy

- Top-level browsing includes products assigned anywhere in that category tree.
- Middle-level browsing includes only the requested subtree, not sibling branches.
- Region comparison is exact and case-sensitive.
- Unknown categories produce zero matches without crashing.
- Product fields are parsed into the correct model members.
- Popularity scores are read and used for ranking.
- Ties are resolved by `product_id` in ascending order.
- `matched_count` is calculated before truncation.
- A missing or null `max_results` value defaults to 5.
- Output request order matches input request order.

10. Running the Exercise

Use the project scripts supplied with the repository. A typical workflow is:

```
./test.sh  
./run.sh
```

Read the failing tests, trace the code path, make the smallest correct changes, and run the tests again. Your final implementation should pass all visible and hidden tests.

Implementation reminder

Treat each JSONL line as one complete JSON object. Preserve the existing public data models and output schema. Focus on correctness first; optimize only after the behavior is correct.

11. Reference Solution - Required Code Changes

The following edits correct the parsing, hierarchy traversal, filtering, and ordering defects. The line numbers refer to the original source files supplied with the exercise.

11.1 `src/browse/parser.cpp`

- Line 84: assign the `product_id` JSON value to `p.product_id`.

- Line 86: assign the name JSON value to p.name.
- Line 92: parse popularity_score instead of skipping it.

```
if (key == "product_id") {
    p.product_id = parse_string();
} else if (key == "name") {
    p.name = parse_string();
} else if (key == "category_id") {
    p.category_id = parse_string();
} else if (key == "region") {
    p.region = parse_string();
} else if (key == "popularity_score") {
    p.popularity_score = parse_int();
} else {
    skip_value();
}
```

11.2 src/browse/engine.cpp

- Lines 16-18: build the parent-to-children adjacency map.

```
for (const auto& category : categories) {
    children[category.category_id];

    if (category.parent_id.has_value()) {
        children[*category.parent_id].push_back(category.category_id);
    }
}
```

- After line 28: return an empty category list when the requested category does not exist.

```
if (children_.find(root_category_id) == children_.end()) {
    return {};
}
```

- Lines 67-71: keep only products in the selected subtree and exact region.

```
for (const auto& product : products) {
    if (included.find(product.category_id) == included.end() ||
        product.region != request.region) {
        continue;
    }

    eligible.push_back(product);
}
```

- Lines 74-79: sort by popularity_score descending, then product_id ascending.

```
std::sort(eligible.begin(), eligible.end(),
    [](const Product& a, const Product& b) {
        if (a.popularity_score != b.popularity_score) {
            return a.popularity_score > b.popularity_score;
        }

        return a.product_id < b.product_id;
    });
```

Compatibility note

The solution uses find() instead of contains() so it works with C++17 and earlier standard-library implementations.